Experiment Number : 8 **Date: 12-02-2026**

Implementation of Packages in Java

Aim:

Write a Java program to implement built-in, user-defined packages and accessing all classes in a package.

PRE LAB EXERCISE

QUESTIONS

1. What is java.util package and what collection framework does it contain?

   java.util is a built-in package in Java that contains utility classes and interfaces for data structures, date & time, random numbers, and more.
   It contains the Java Collection Framework, which provides classes and interfaces to store and manipulate groups of objects.
   Main interfaces in Collection Framework:

- List
- Set
- Queue
- Map

Common classes:

- ArrayList
- LinkedList
- HashSet
- HashMap

2. What are the two types of packages in Java?

   There are two types of packages:

1. **Built-in Packages** – Already provided by Java (example: java.lang, java.util).
2. **User-defined Packages** – Created by the programmer using the package keyword to organize classes.

IN LAB EXERCISE

Objective

To understand and implement the concepts of built-in, user-defined packages and accessing all classes in a package in Java.

Built-in Packages comprise a large number of classes that are part of the Java API. Some of the commonly used built-in packages are:

- java.lang: Contains language support classes(e.g, classes that define primitive data types, math operations). This package is automatically imported.
- java.io: Contains classes for supporting input/output operations.
- java.util: Contains utility classes that implement data structures such as Linked Lists and Dictionaries, as well as support for date and time operations.
- java.applet: Contains classes for creating Applets.
- java.awt: Contains classes for implementing the components for graphical user interfaces (like buttons, menus, etc).

Source Code

```
import java.util.Random;   // built-in package

public class Sample{

      public static void main(String[] args) {

        // using Random class

       Random rand = new Random();

       // generates a number between 0–99

       int number = rand.nextInt(100);

       System.out.println("Random number: " + number);

    }

}
```

Output

Random number: 49

```
Random number: 35
```

User-defined Packages are the packages that are defined by the user.

Source code

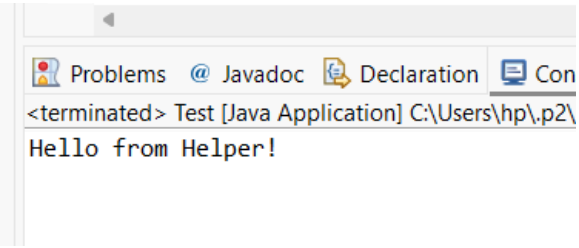
```
package com.myapp;
public class Helper {
   public static void show() {
      System.out.println("Hello from Helper!");
   }
}
```

==To use this in another class==

```
import com.myapp.Helper;
public class Test {
   public static void main(String[] args) {
      Helper.show();
   }
}
```

Output:

Hello from Helper!



//Importing all classes from a package.

Source code

```
import java.util.Vector;
public class Coders {
    public Coders() {
```

```
        // java.util.Vector is imported, We are able to access it directly in our code.
    Vector v = new Vector();
    java.util.ArrayList l = new java.util.ArrayList();
    l.add(3);
    l.add(5);
    l.add(7);
    System.out.println(l);
  }
  public static void main(String[] args) {
    new Coders();
  }
}
```

Output

[3,5,7]

POST LAB EXERCISE

1. What will happen if two classes in different packages have the same name and are imported in a Java file?

   If two classes with the same name are imported from different packages, it creates an **ambiguity error (compile-time error)**.
   The compiler will not know which class to use.
   To solve this, we must use the **fully qualified class name** (package name + class name).
   Example:
   java.util.Date
   java.sql.Date

2. What is the purpose of using packages in Java?

   Packages are used to:

- Organize related classes and interfaces

- Avoid name conflicts
- Provide access protection
- Improve code readability and maintenance
  They help manage large programs easily.

3. Which built-in Java package would you use if you want to create a GUI window and display a message?
   A. java.util
   B. java.sql
   C. java.awt
   D. java.net

   Correct Answer: C. `java.awt`

   `java.awt` (Abstract Window Toolkit) is used to create GUI components like windows, buttons, labels, etc.

ASSESSMENT

Description	Max Marks	Marks Awarded
Pre Lab Exercise	**5**	
In Lab Exercise	**10**	
Post Lab Exercise	**5**	
Viva	**10**	
Total	**30**	
Faculty Signature		

Experiment Number : 9 **Date: 12-02-2026**

Implementation of a Java Program to import packages using different methods

Aim:

Write a Java program to import packages using different methods for different use cases.

PRE LAB EXERCISE

QUESTIONS

1. How to import a single class and multiple classes from a package in Java?

How to import a single class and multiple classes from a package in Java?

- To import a **single class**, use:
  import packageName.ClassName;
  Example:
  import java.util.Scanner;
- To import **all classes** from a package, use *:
  import packageName.*;
  Example:
  import java.util.*;
  * imports all classes of that package (but not sub-packages).

2. Which package is always imported by default in every Java class?
   The java.lang package is automatically imported by default.
   It contains important classes like:

- String
- System
- Math
- Object

  That is why we do not need to write:
  import java.lang.*;

IN LAB EXERCISE

Objective

To understand and implement the Java packages using different methods and import them.

Problem

Define a package named ‘useFul’ with a class names ‘UseMe’ having following methods:

1) area()- To calculate the area of given shape.

2) salary()- To calculate the salary given basic Salary,da,hRA.

3) percentage()-To calculate the percentage given total marks and marks obtained.

4) Develop a program named ‘Package Use’ to import the above package ‘useFul’ and use the method area().

5) Develop a program named ‘manager’

Source Code

```
//Package Creation:

package useFull;

import java.util.*;

public class UseMe

{

        Scanner obj=new Scanner(System.in);

public static void area()

      {

          class method{

            void aos(int a)

            {

              System.out.print("\nArea of square with length "+a+" is "+(a*a));

            }

           void aor(int a,int b)

           {

              System.out.print("\nArea of reactangle with dimensions "+a+" & "+b+" is "+(a*b));

            }

            void aoc(int r)

           {

             double a=3.14*r*r;

            }

           System.out.print("\nArea of circle with radius "+r+" is "+a);
```

```
}
void aot(int a,int b)
 {
   float ar=(a*b)/2;
   System.out.print("\nArea of triangle with dimensions "+a+" &"+b+" is "+ar);
  } }
Scanner obj=new Scanner(System.in);
method m=new method();
System.out.print("\n1.Square\n2.Rectangle\n3.Circle\n4.Triangle\nSelect the shape\n");
int ch=obj.nextInt();
UseMe u=new UseMe();
switch(ch)
        {
        case 1:System.out.print("\nEnter the length of side of square : ");
                int s=obj.nextInt();m.aos(s);
                break;
        case 2:System.out.print("\nEnter the dimensions of rectangle : ");
                int l=obj.nextInt();
                int b=obj.nextInt();
                m.aor(l,b);
                break;
        case 3:System.out.print("\nEnter the radius of circle : ");
                int r=obj.nextInt();
                m.aoc(r);
                break;
        case 4:System.out.print("\nEnter the dimensions of triangle : ");
                int ba=obj.nextInt();
                int w=obj.nextInt();
```

```
                    m.aot(ba,w);
                    break; } }
public void salary()
  {
        int ba,da,hra;
        System.out.print("\nEnter the basic salary : ");
        ba=obj.nextInt();
        System.out.print("\nEnter the dearness allowance :");
        da=obj.nextInt();
        System.out.print("\nEnter the house rent allowance : ");
        hra=obj.nextInt();
        System.out.print("\nThe total Gross salary of employee is : "+(ba+da+hra));
   }
public void percentage()
   {
        int n,sum=0;
        float p;
        System.out.print("\nEnter the total number of subjects : ");
        n=obj.nextInt();
        int m[]=new int[n];
        System.out.print("\nEnter the marks of "+n+" subjects : ");
        for(int i=0;i<n;i++)
        {
           m[n]=obj.nextInt();
        }
        for(int i=0;i<n;i++)
        {
            sum=sum+m[i];
```

```
        }
        p=sum/n;
        {
                System.out.print("\nPercentahe of student : "+p);
        }
    }
}
//Package Implementation-1:
        import useFull.UseMe;
        class packageUse
        {
                public static void main(String args[])
                {
                UseMe o=new UseMe();o.area();
                }
        }
```

Output

javac packageUse.java

java packageUse

1. Square
2. Rectangle
3. Circle
4. Triangle

Select the shape
2

Enter the dimensions of the rectangle: 10 15
Area of the rectangle with dimensions 10&15 is 150

```
1. Square
2. Rectangle
3. Circle
4. Triangle
Select the shape: 4
Enter base: 3
Enter height: 5
Area of Triangle = 7.5
```

//Package Implementation-2:

```
import useFull.UseMe;
class manager
{
   public static void main(String args[])
   {
    UseMe obj=new UseMe();obj.salary();
    }
}
```

Output

```
Enter Basic Salary: 100000
Enter Dearness Allowance: 5000
Enter House Rent Allowance: 2000
Total Gross Salary = 107000
```

javac manager.java

java manager

Enter the basic salary: 100000

Enter the dearness allowance: 5000

Enter the house rent allowance: 2000

The total Gross salary of employee is: 107000

POST LAB EXERCISE

1. Find the key differences between java.util and java.lang packages.

`java.lang`	`java.util`
Automatically imported by default	Must be imported manually
Contains basic core classes	Contains utility and collection classes
Used for fundamental operations	Used for data structures and helper utilities
Examples: `String`, `System`, `Math`, `Object`	Examples: `Scanner`, `ArrayList`, `HashMap`, `Date`

2. List some of the subpackages of java.util

Some important subpackages are:

- java.util.concurrent (multithreading utilities)
- java.util.function (functional interfaces)
- java.util.logging (logging API)
- java.util.regex (regular expressions)
- java.util.stream (stream API for data processing)
- java.util.zip (ZIP file handling)

ASSESSMENT

Description	Max Marks	Marks Awarded
Pre Lab Exercise	**5**	
In Lab Exercise	**10**	
Post Lab Exercise	**5**	
Viva	**10**	
Total	**30**	
Faculty Signature		

Experiment Number : 10 **Date: 12-02-2026**

Implementation of Access Modifiers in Java

Aim:

Write a Java program to implement different access modifiers.

PRE LAB EXERCISE

QUESTIONS

1. What are the 4 access modifiers available in Java?
   The four access modifiers in Java are:
1. private – Accessible only within the same class
2. default (no modifier) – Accessible only within the same package
3. protected – Accessible within the same package and in subclasses
4. public – Accessible from anywhere

2. Which access modifiers can be used in Java to control access to class members?
   Class members (variables, methods) can use **all four modifiers**:

- private → most restricted
- default → package-level access
- protected → package + subclass access
- public → unrestricted access

IN LAB EXERCISE

Objective

To demonstrate different access modifiers such as Default, Private, Protected and Public using Java programs.

	Default	Private	Protected	Public
Same Class	Yes	Yes	Yes	Yes
Same Package Subclass	Yes	No	Yes	Yes
Same Package Non-Subclass	Yes	No	Yes	Yes
Different Package Subclass	No	No	Yes	Yes
Different Package Non-Subclass	No	No	No	Yes

Fig: Comparison table of Access Modifiers in Java

Source Code

// Default Access modifier

```
class Car {
   String model; // default access
}
public class Main {
      public static void main(String[] args){
       Car c = new Car();
       c.model = "Tesla"; // accessible within the same package
       System.out.println(c.model);
   }
}
```

Output:

Tesla

```
<terminated> Main (1) [Java Application] C:\User
Tesla
```

//Private Access Modifier

```
class Person {
```

```
    // private variable
    private String name;
    public void setName(String name)  {
     this.name = name; // accessible within class
    }
    public String getName() {
            return name;
   }
}
public class Geeks {
    public static void main(String[] args)
    {
       Person p = new Person();
        p.setName("Alice");


        // System.out.println(p.name);
        // Error: 'name'
        // has private access
        System.out.println(p.getName());
    }
}
```

Output

Alice

```
<terminated> Geeks [Java Application] C:\Users\sa
Alice
```

//Protected Access Modifier

```
class Vehicle {
    protected int speed; // protected member
}
class Bike extends Vehicle {
    void setSpeed(int s)
    {
        speed = s; // accessible in subclass
    }
    int getSpeed()
    {
        return speed; // accessible in subclass
    }
}
public class Main {
    public static void main(String[] args){
        Bike b = new Bike();
        b.setSpeed(100);
        System.out.println("Access via subclass method: "+ b.getSpeed());
        Vehicle v = new Vehicle();
        System.out.println(v.speed);
    }
}
```

Output

Access via subclass method: 100

0

```
Access via subclass method: 100
0
```

//Public Access Modifier

```
class MathUtils {
    public static int add(int a, int b) {
    return a + b;
  }
}
public class Main {
    public static void main(String[] args) {
    System.out.println(MathUtils.add(5, 10)); // accessible anywhere
  }
}
```

Output

15

```
<terminated> Main (1) [Java Application] C:\U
15
```

POST LAB EXERCISE

1. Write a Java program to implement Default access modifier which has a default class within the same package and a default class from a different package.

   Default Access Modifier in Java
   If no access modifier is specified for a class, method, or variable, it is called the default access modifier.
   It is accessible only within the same package and not accessible from another package.

   1. Accessing Default Class Within the Same Package
   File 1: DefaultClass.java

```
package pack1;

// default class (no public keyword)
class DefaultClass {

    void display() {
        System.out.println("Default class method accessed inside same package");
    }
}
```

File 2: SamePackageTest.java

```
package pack1;

public class SamePackageTest {

    public static void main(String[] args) {

        DefaultClass obj = new DefaultClass();
        obj.display();

    }
}
```

Output

Default class method accessed inside same package

Explanation

- Both classes are in pack1
- Default class is accessible.

2. Accessing Default Class From Different Package

File 3: DifferentPackageTest.java

```
package pack2;

import pack1.DefaultClass;   // trying to import

public class DifferentPackageTest {

    public static void main(String[] args) {

        DefaultClass obj = new DefaultClass();
        obj.display();

    }
}
```

Output

Error: DefaultClass is not public in pack1; cannot be accessed from outside package

Explanation

- Default class cannot be accessed outside the package.

2. Which access modifier provides the highest level of access?
   Answer: public

Reason:

- Public members or classes are accessible from anywhere, both within the same package and from other packages.
- There is no restriction on who can use them.
- Can be accessed by any class in the project, library, or external packages.
- Commonly used for API methods, main classes, and interfaces that need to be widely accessible.
- Provides maximum visibility, so it should be used carefully to avoid unintended access or misuse.

ASSESSMENT

Description	Max Marks	Marks Awarded
Pre Lab Exercise	**5**	
In Lab Exercise	**10**	
Post Lab Exercise	**5**	
Viva	**10**	
Total	**30**	
	Faculty Signature	

Experiment Number :11 **Date: 12-02-2026**

Implementation of multiple inheritance in Java using Interface

Aim:

Write a Java program to implement multiple inheritance using Interface.

PRE LAB EXERCISE

QUESTIONS

1. Why Java does not support multiple inheritance using classes like that of C?

Java does not support multiple inheritance with classes to avoid ambiguity problems, especially the Diamond Problem.

Example of the issue:

If Class A has a method display(),
Class B and Class C inherit from A,
and Class D inherits from both B and C.

Now when D calls display(), the compiler cannot determine whether it should use the method from B or C.

This creates confusion and complexity in the program.

To avoid this problem and keep Java simple and reliable, Java does not allow multiple inheritance through classes.
Instead, Java supports multiple inheritance using interfaces.

2. What is the primary purpose of an interface in Java?

A. To store data using instance variables

B. To define a contract of methods a class must implement

D. To allow creation of objects

D. To improve runtime performance

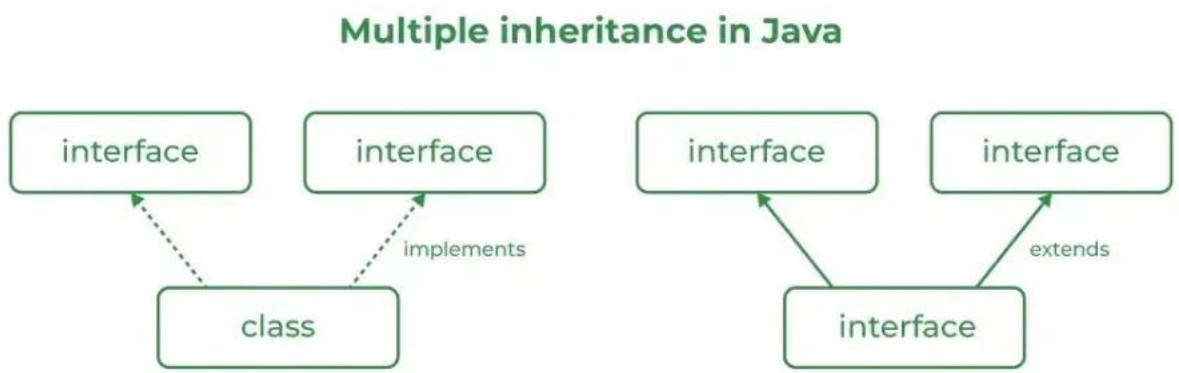
Correct Answer:
B. To define a contract of methods a class must implement

Explanation:
An interface in Java contains method declarations. Any class that implements the interface must provide implementations for those methods. This ensures a common structure for different classes.

IN LAB EXERCISE

Objective

To demonstrate how an interface in Java defines constants and abstract methods, which are implemented by a class.



Source Code

```
import java.io.*;
  // Add interface
interface Add{
    int add(int a,int b);
  }
// Sub interface
interface Sub{
            int sub(int a,int b);
  }
// Calculator class implementing Add and Sub
```

```
class Cal implements Add , Sub
  {
// Method to add two numbers
        public int add(int a,int b){
        return a+b;
   }
        // Method to sub two numbers
        public int sub(int a,int b){
        return a-b;
   }
}
class Example{
   // Main Method
   public static void main (String[] args){

        // instance of Cal class
        Cal x = new Cal();
        System.out.println("Addition : " + x.add(2,1));
        System.out.println("Substraction : " + x.sub(2,1));
   }
}
```

Outputs

Addition : 3
Subtraction : 1

```
Addition : 3
Substraction : 1
```

POST LAB EXERCISE

1. Can a functional interface extend another interface?
   Yes, a functional interface can extend another interface, but only if it still has exactly one abstract method after inheritance.
   If extending another interface introduces more than one abstract method, then it will no longer be a functional interface.
   Example:
   ```
   interface A {
      void show();
   }

   FunctionalInterface
   interface B extends A {
      // still only one abstract method
   }
   ```
   Here, B is still a functional interface because there is only one abstract method.

2. Which feature was introduced in interfaces starting from Java 8?

   A. Constructors

   B. Private methods

   C. Default and static methods

   D. Final classes

   Correct Answer:
   C. Default and static methods
   Explanation:
   Starting from Java 8, interfaces can contain:
   - Default methods (with method body)
   - Static methods

   This was introduced to allow new methods to be added to interfaces without breaking existing implementations.

ASSESSMENT

Description	Max Marks	Marks Awarded
Pre Lab Exercise	5	
In Lab Exercise	10	
Post Lab Exercise	5	
Viva	10	
Total	**30**	
	Faculty Signature	

Experiment Number : 12 **Date: 25-02-2026**

Implementation of exception handling in Java

Aim:

Write a java program to implement exception handling.

PRE LAB EXERCISE

QUESTIONS

1. What is exception handling in Java?
   Exception handling in Java is a mechanism used to handle runtime errors so that the normal flow of the program can be maintained. Instead of stopping the program when an error occurs, Java provides constructs like try, catch, finally, throw, and throws to detect and manage errors properly.
   It helps in preventing program crashes and allows the programmer to handle unexpected situations such as division by zero, file errors, or invalid input.

2. What is the purpose of using try-catch blocks in exception handling?
   The try-catch block is used to handle exceptions that may occur during program execution.
   - The try block contains code that might cause an exception.
   - The catch block handles the exception if it occurs.

   This ensures that the program does not terminate abruptly and allows the developer to display an error message or take corrective action.

IN LAB EXERCISE

Objective

To understand and implement exception handling through try-catch and finally blocks.

Source Code

```
import java.util.InputMismatchException;

import java.util.Scanner;

public class ExceptionHandlingExample {

 public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

         try {

        System.out.print("Enter an integer: ");

        int num = scanner.nextInt();

         int result = 10 / num;

        System.out.println("Result: " + result);

        } catch (InputMismatchException e) {
```

```
        System.out.println("Invalid input! Please enter an integer.");
        } catch (ArithmeticException e) {
        System.out.println("Cannot divide by zero!");
        } finally {
        scanner.close();
        System.out.println("Program execution completed.");
        } }
}
```

Outputs

Output 1 (Valid Input - Non-zero Integer):

Enter an integer: 5

Result: 2

Program execution completed.

```
<terminated> ExceptionHandlingExample [Java Applic
Enter an integer: 5
Result: 2
Program execution completed.
```

Output 2 (Valid Input - Zero):

Enter an integer: 0

Cannot divide by zero!

Program execution completed.

```
<terminated> ExceptionHandlingExample [Java Applicatio
Enter an integer: 0
Cannot divide by zero!
Program execution completed.
```

Output 3 (Invalid Input - Non-integer):

Enter an integer: abc

Invalid input! Please enter an integer.

Program execution completed.

```
<terminated> ExceptionHandlingExample [Java Applicatio
Enter an integer: sa
Invalid input! Please enter an integer.
Program execution completed.
```

POST LAB EXERCISE

1. What is the purpose of the Scanner object in the Java program?
   The Scanner object is used to take input from the user. It reads data entered from the keyboard (System.in) such as integers, strings, floats, etc., and allows the program to use that input for processing.
   Example: reading numbers, text, or other values from the user.

2. What exceptions are expected to be thrown in the code within the try block? Why?
   Common exceptions that may occur inside the try block include:

- ArithmeticException – occurs when an illegal arithmetic operation happens, such as dividing a number by zero.
- InputMismatchException – occurs when the user enters input of the wrong data type (for example, entering text when the program expects a number).
- ArrayIndexOutOfBoundsException – occurs when the program tries to access an array index that does not exist.
  These exceptions occur due to invalid operations or incorrect user input, so they are handled using try-catch blocks to prevent the program from crashing.

ASSESSMENT

Description	Max Marks	Marks Awarded
Pre Lab Exercise	5	
In Lab Exercise	10	
Post Lab Exercise	5	
Viva	10	
Total	**30**	
	Faculty Signature	

Experiment Number : 13 **Date: 25-02-2026**

Implementation of User-defined Exception in Java

Aim:

Write a java program to implement User-defined Exception.

PRE LAB EXERCISE

QUESTIONS

1. What is Java custom exception?
   A custom exception in Java is a user-defined exception created by the programmer to handle specific errors in an application.
   It is created by extending the Exception class or RuntimeException class. Custom exceptions help make programs more meaningful and easier to debug when special conditions occur.
   Example situations:
   - Invalid age
   - Insufficient balance
   - Invalid login

2. What are the two types of custom exceptions in Java?
   - **Checked Custom Exception**
     - Created by extending the Exception class.
     - The compiler checks these exceptions at compile time.
     - They must be handled using try-catch or throws.
   - **Unchecked Custom Exception**
     - Created by extending the RuntimeException class.
     - These are checked at runtime and handling them is optional.

IN LAB EXERCISE

Objective

To understand and implement Checked and Unchecked custom exceptions.

Source Code

Ex 1: // Custom Checked Exception

```
class InvalidAgeException extends Exception {
   public InvalidAgeException(String m) {
      super(m);
   }
}
// Using the Custom Exception
public class AgeCheck{
```

```
    public static void validate(int age)
      throws InvalidAgeException {
        if (age < 18) {
            throw new InvalidAgeException("Age must be 18 or above.");
        }
        System.out.println("Valid age: " + age);
    }
public static void main(String[] args) {
  try {
            validate(12);
        } catch (InvalidAgeException e) {
            System.out.println("Caught Exception: " + e.getMessage());
        }
    }
}
```

Output 1

Caught Exception: Age must be 18 or above.

```
Caught Exception: Age must be 18 or above.
```

Ex 2: // Custom Unchecked Exception

```
class DivideByZeroException extends RuntimeException {
    public DivideByZeroException(String m) {
        super(m);
    }
}
// Using the Custom Exception
```

```
public class TestSample {
   public static void divide(int a, int b) {
      if (b == 0) {
         throw new DivideByZeroException("Division by zero is not allowed.");
      }
      System.out.println("Result: " + (a / b));
   }
public static void main(String[] args) {
  try {
         divide(10, 0);
      } catch (DivideByZeroException e) {
         System.out.println("Caught Exception: " + e.getMessage());
      }
   }
}
```

Output 2

Caught Exception: Division by zero is not allowed.

```
Caught Exception: Division by zero is not allowed.
```

POST LAB EXERCISE

1. What is required to create a custom exception in Java?
   A. Extend the Object class
   B. Extend the Throwable class
   C. Extend Exception or RuntimeException
   D. Implement the Serializable interface

   Correct Answer:
   C. Extend Exception or RuntimeException

Explanation:
To create a custom exception in Java, a class must extend either the Exception class (for checked exceptions) or the RuntimeException class (for unchecked exceptions)

2. List some use cases for the checked and unchecked exceptions.

Checked Exceptions (checked at compile time):

- File handling errors (File not found)
- Database connection errors
- Network connection issues
- Reading or writing files

These must be handled using try-catch or throws.

Unchecked Exceptions (occur at runtime):

- Division by zero (ArithmeticException)
- Invalid array index (ArrayIndexOutOfBoundsException)
- Null object access (NullPointerException)
- Invalid input errors

These occur due to programming mistakes or logical errors and handling them is optional.

ASSESSMENT

Description	Max Marks	Marks Awarded
Pre Lab Exercise	**5**	
In Lab Exercise	**10**	
Post Lab Exercise	**5**	
Viva	**10**	
Total	**30**	
	Faculty Signature	